

Rapport technique – Implémentation d'un système RAG pour la recommandation d'événements culturels

```
Auteur : Samir Belasri  
Date : Avril 2026  
Statut : Preuve de Concept (PoC)  
$ _
```

 Python 3.13

 FAISS

 Mistral AI

 LangChain

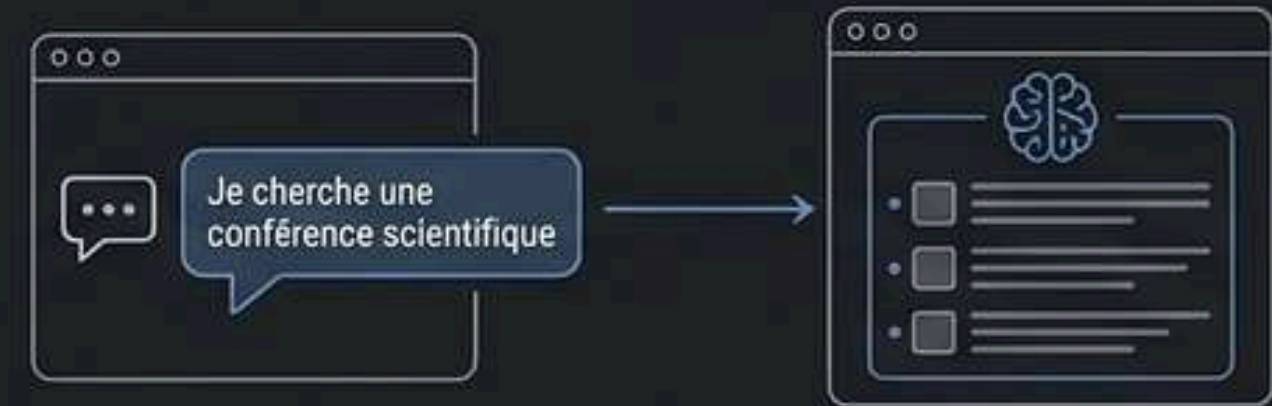
Surabondance de données non structurées

Les données d'événements culturels sont riches mais complexes à explorer efficacement via les méthodes traditionnelles.



Recommandation dynamique en langage naturel

Permettre aux utilisateurs de poser des requêtes complexes (ex: "Je cherche une conférence scientifique") et d'obtenir des recommandations contextualisées.



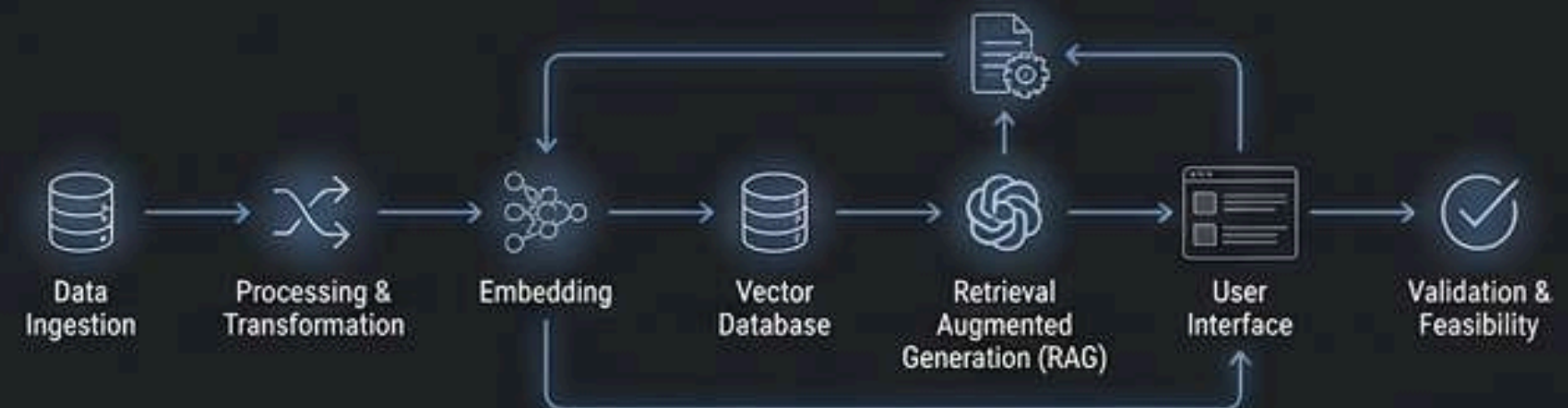
Dépasser la recherche par mots-clés

Comment interroger sémantiquement une base vectorielle d'événements sans dépendre de requêtes SQL rigides ou de correspondances exactes ?



Preuve de concept Data Engineering

Ce projet valide la faisabilité technique de bout en bout du pipeline data et de l'architecture RAG.



projet_11_rag_events/

|— src/

|— data/

|— tests/

|— .env.example

|— requirements.txt

Isolation

J'utilise un environnement virtuel (.venv) couplé à un fichier requirements.txt strict pour figer les dépendances et garantir la reproductibilité.

Sécurité

J'injecte les secrets (clés API OpenAgenda et Mistral) via un fichier .env. Je justifie l'exclusion de ce fichier du dépôt via un .gitignore rigoureux.

Exécution

Le pipeline est conçu pour être lancé de bout en bout via une série de scripts modulaires.

OpenDataSoft

✓ **Avantage** : Dataset consolidé.

✗ **Critique technique** :
Présence d'anomalies bloquantes sur
les champs temporels. J'ai constaté
des dates aberrantes (ex: événements
fixés à l'année 2500).



API OpenAgenda

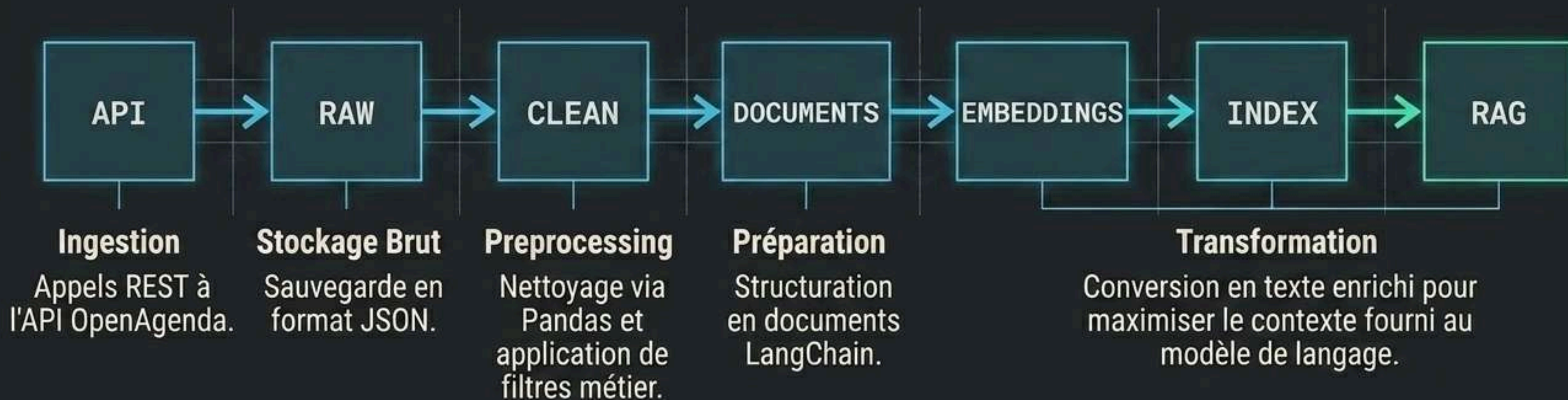
✓ **Avantage** : Accès
programmatique direct,
requêtes paramétrables,
données fraîches.



Justification du choix final

J'ai sélectionné l'API OpenAgenda pour garantir l'intégrité temporelle absolue des événements, un critère non négociable pour un système de recommandation.





Nettoyage brut

Utilisation de Pandas pour la suppression des valeurs nulles et le traitement des données manquantes.

Gestion des dates

Parsing rigoureux des formats temporels pour écarter les anomalies.

Filtres métier

Filtres métier

J'ai appliqué des filtres géographiques stricts (Périmètre Paris-Saclay incluant Paris, Gif-sur-Yvette, Palaiseau) et conservé uniquement les événements récents ou futurs.

Résultat

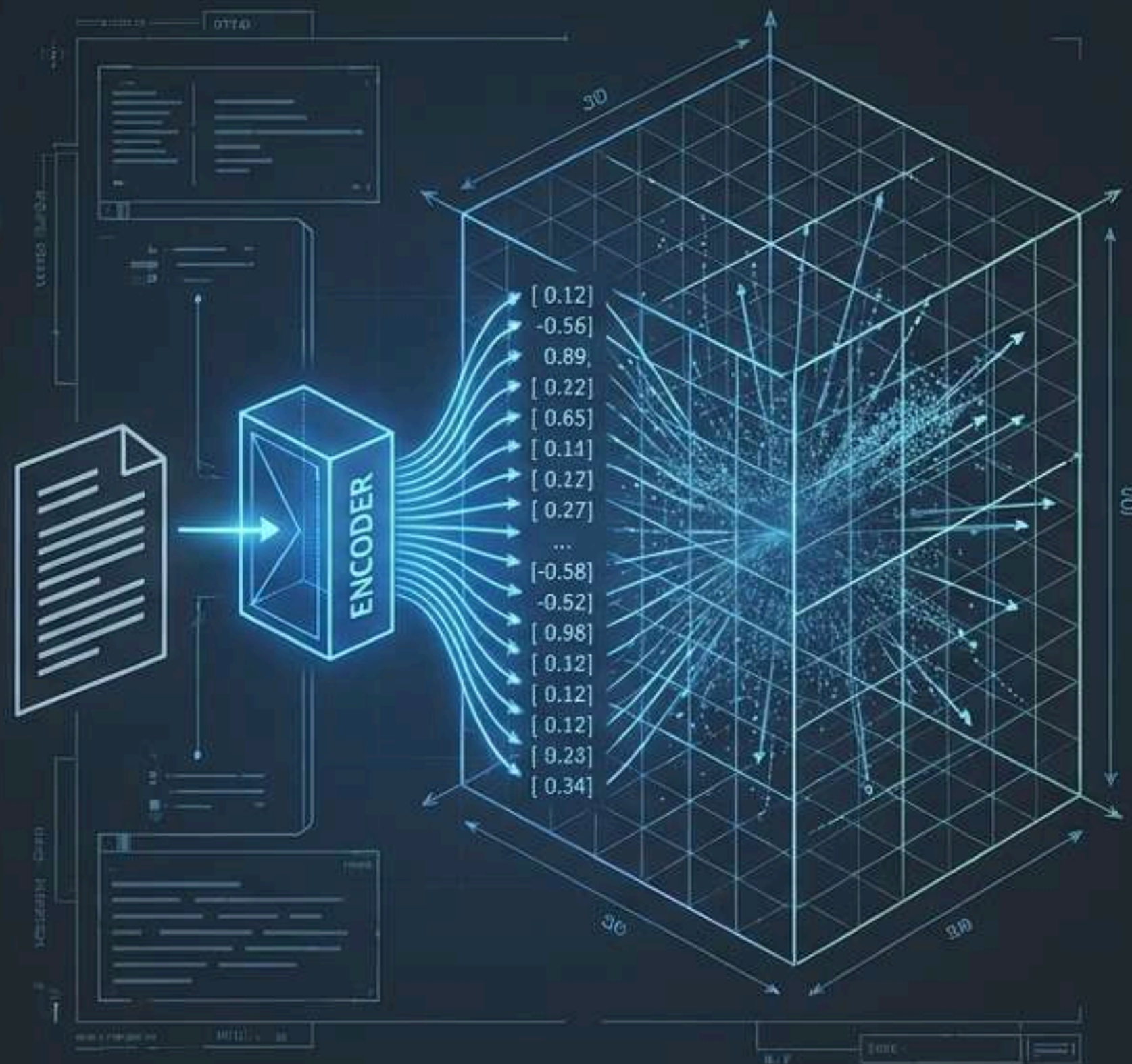
Un dataset sain (events_clean.csv), débarrassé de ses anomalies, prêt à être ingéré par le LLM sans risque d'hallucination due au bruit.

Principe des Embeddings

Je transforme la sémantique du texte en représentations vectorielles denses.

Modèle Utilisé

J'ai sélectionné Mistral AI pour sa performance reconnue sur la génération d'embeddings en langue française.



Indexation

Utilisation de FAISS (Facebook AI Similarity Search).

Justification Technique

J'ai choisi FAISS pour sa capacité à exécuter des recherches de similarité (L2/Cosine) ultra-rapides en local. L'index est sérialisé et stocké localement, m'affranchissant d'une dépendance cloud externe.

1. Transformation de la question

La requête utilisateur est convertie en embedding dense via Mistral.

2. Recherche sémantique

L'index FAISS calcule la distance entre le vecteur de la question et les vecteurs des événements.

3. Sélection

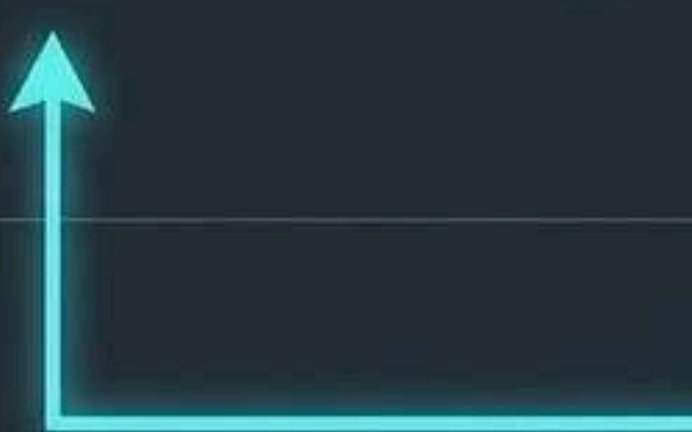
Extraction des k documents les plus mathématiquement similaires.

4. Ingénierie du Prompt

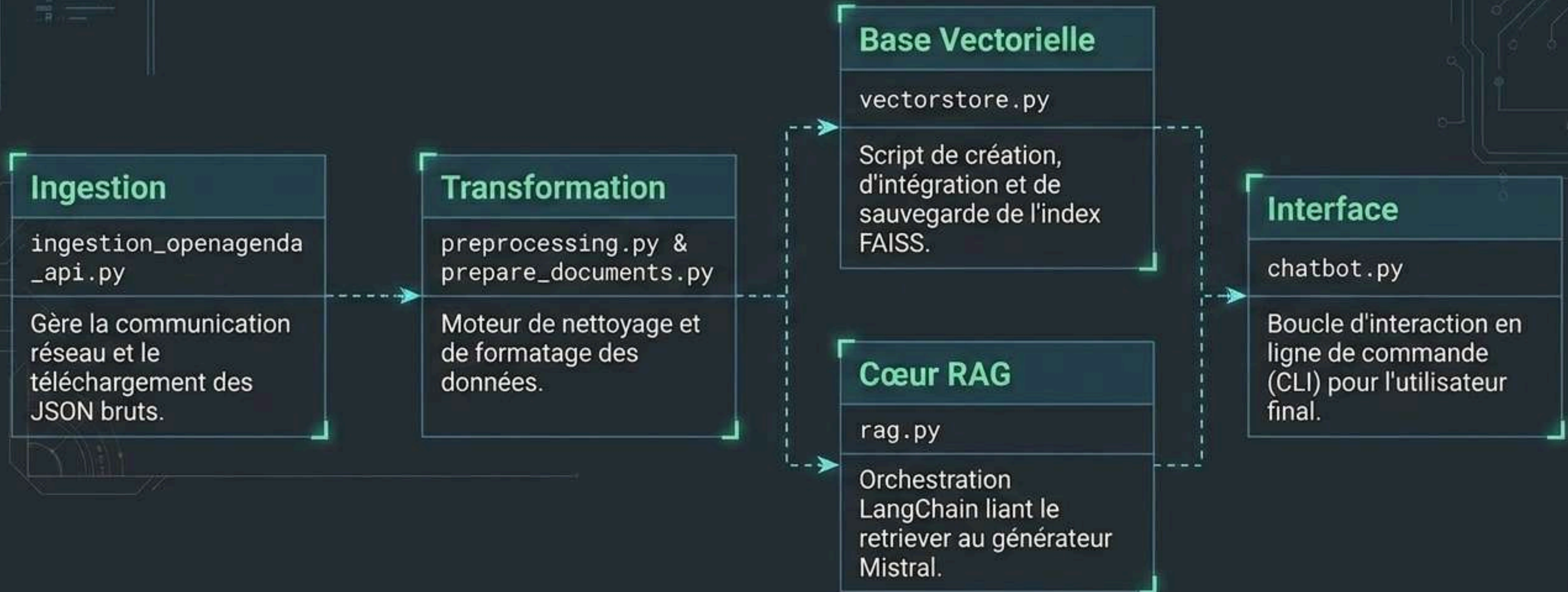
Je construis un prompt strict combinant la question initiale et le contexte textuel des k documents récupérés.

5. Génération

Appel au LLM Mistral AI pour inférer une réponse naturalisée, structurée et ancrée exclusivement dans le contexte fourni.



Architecture Logique de l'Implémentation



Stratégie de Test du Pipeline

Validation Métier

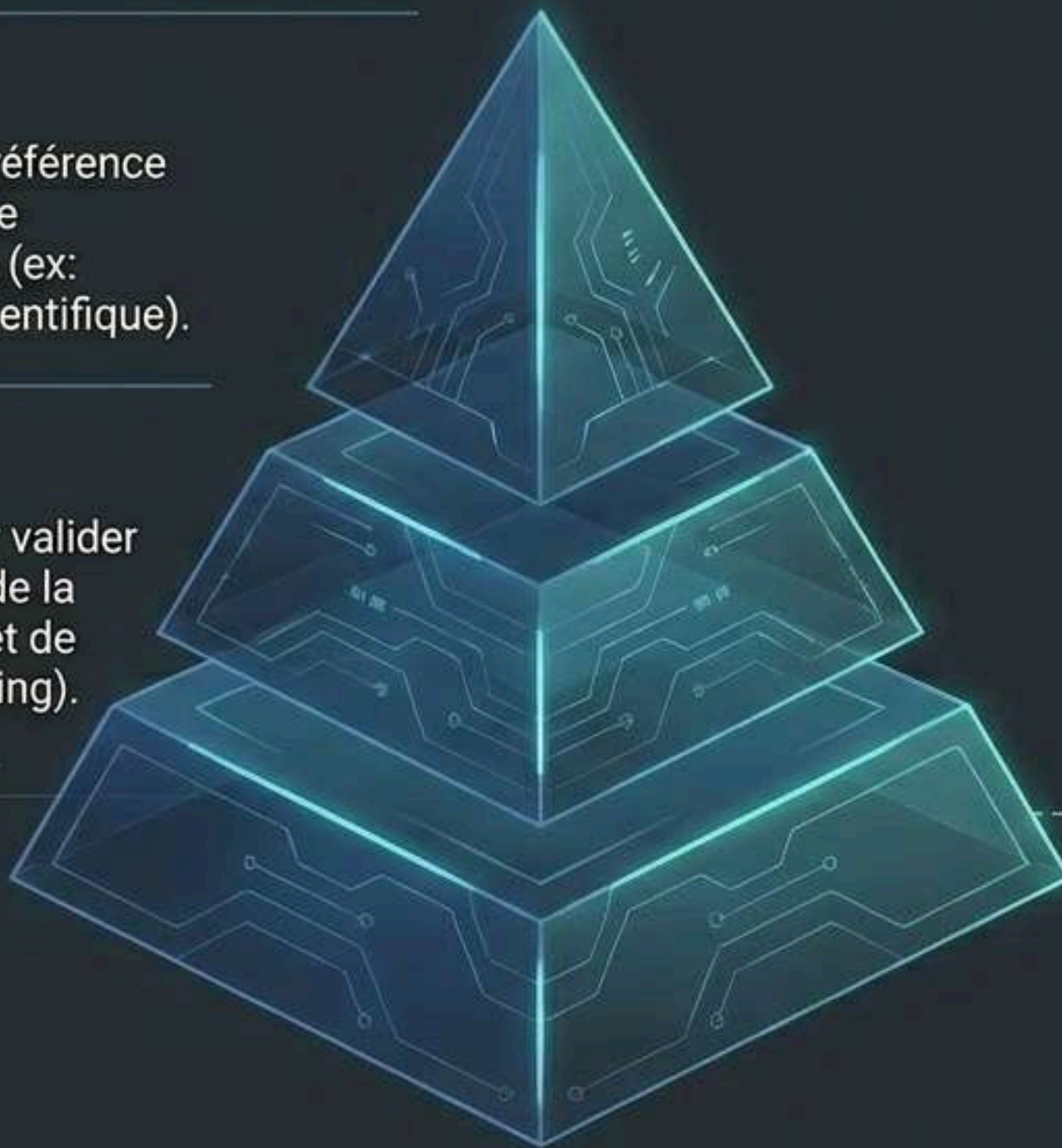
Conception d'un jeu de données de référence (test_qa.json) contenant 12 paires de questions/réponses représentatives (ex: exposition artistique, conférence scientifique).

Data Quality

Création de test_data_quality.py pour valider l'intégrité métier (vérification stricte de la présence des colonnes obligatoires et de la validité des dates post-preprocessing).

Tests Unitaires

Utilisation du framework pytest pour valider isolément les fonctions critiques du code.



Évaluation Automatisée

Le script evaluate_rag.py confronte le pipeline complet à ce jeu de test pour vérifier la robustesse du système.

Évaluation Qualitative & Limites du Système

Méthodologie Employée



- Mon évaluation repose sur une analyse qualitative experte.



- J'inspecte visuellement la pertinence des réponses du modèle, la présence des informations clés (lieu, date) et la cohérence face au référentiel test_qa.json.



Limites Assumées



- J'explique explicitement l'absence de métriques automatiques de scoring quantitatif.



- Le jugement reste heuristique. Bien que fonctionnel pour un PoC, le système ne permet pas encore de quantifier la dérive (*drift*) ou la précision mathématique exacte des réponses au-delà de la validation visuelle de l'ingénieur.

Analyse Critique du PoC



Limites du Dataset

Le volume de données est restreint au périmètre géographique choisi, limitant la couverture des cas d'usage exhaustifs.



Limites du RAG (Pas de Chunking)

J'ai fait le choix technique de ne pas utiliser de découpage (chunking), car les descriptions d'événements actuelles étaient suffisamment courtes pour tenir intégralement dans la fenêtre de contexte. Limite assumée si les textes s'allongent.



Limites de l'Interface

L'interaction utilisateur est actuellement restreinte à une ligne de commande (CLI), limitant l'accessibilité pour un profil non technique.

Améliorations Possibles (Roadmap V2)



Évaluation (Ajout Métriques)

Intégration du framework RAGAS pour calculer automatiquement la fidélité (Faithfulness) et la pertinence (Answer Relevance).



Scalabilité (Chunking)

Implémentation d'un TextSplitter avancé pour absorber des contenus plus volumineux (PDF, programmes longs).



Backend (API REST)

Exposition du RAG via une API REST (FastAPI) pour dissocier l'infrastructure métier.



Frontend (Interface Web)

Création d'une interface utilisateur riche via Streamlit ou Gradio pour démocratiser l'accès au système.

Références Bibliographiques et Techniques

Architecture RAG : Lewis et al. (2020)

<https://arxiv.org/abs/2005.11401>

Indexation Vectorielle : FAISS Meta Research

<https://github.com/facebookresearch/faiss>

Génération et Embeddings : Mistral AI

<https://docs.mistral.ai>

Orchestration : LangChain Framework

<https://python.langchain.com>

Modèles Dense Encoders : Sentence-Transformers

<https://www.sbert.net>

Source de Données : API OpenAgenda

<https://developers.openagenda.com/>